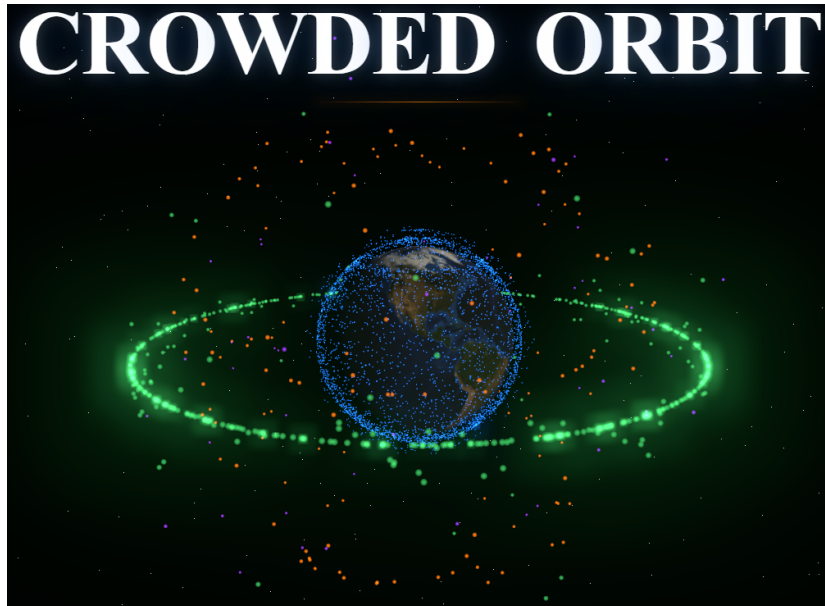


Crowded Orbit

A scrollytelling tour of every operational satellite around Earth.

Kevin Abou Jaoude • Youssef Dib • Mark Nabbout

<https://com-480-data-visualization.github.io/The-Outliers/>



What this project is

Earth's orbit looks empty from below. It is in fact one of the most crowded, unequally controlled pieces of modern infrastructure: 6,713 active satellites at the start of 2023, more than three quarters of them launched in the last four years, half of them operated by a single company, and another tens of thousands already filed with regulators for the coming decade. *Crowded Orbit* makes that transformation visible.

The site is a scrolling, four-chapter narrative aimed at a non-specialist audience (students, tech-aware readers, journalists). It opens on a cinematic 3D scene of the orbital population, walks the reader through the post-2019 explosion, names the operators and countries behind it, drills into where the congestion concentrates inside the 3D globe, and closes on the 2030 projections. Every chart is real-data interactive, not decoration.

Three numbers that drive the story.

- **75.5%** of every active satellite was launched since 2019.
- One company (SpaceX) controls $\approx 50\%$ of all satellites; the operator-distribution Gini is **0.862**.
- Regulators have already approved $\approx 80,000$ satellites for deployment by 2030, a $12\times$ expansion in seven years.

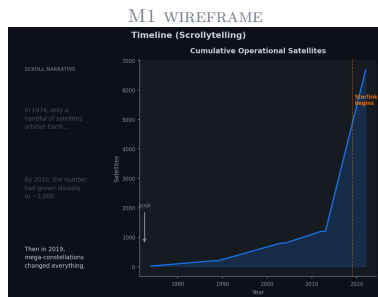
How to read this book

This document is the design log behind that site. Page 2 recaps the path from the original M1 problematic to the final layout (with the M1 wireframes side-by-side with the actual screens). Page 3 explains the data and the cleaning decisions baked into `build_data.py`. Pages 4 and 5 are the narrative-design and visual-design rationale. Pages 6 and 7 cover the interactions, the challenges we worked through, and the technical architecture. Page 8 is the per-member contribution breakdown and our honest reflection on what we would do differently with more time.

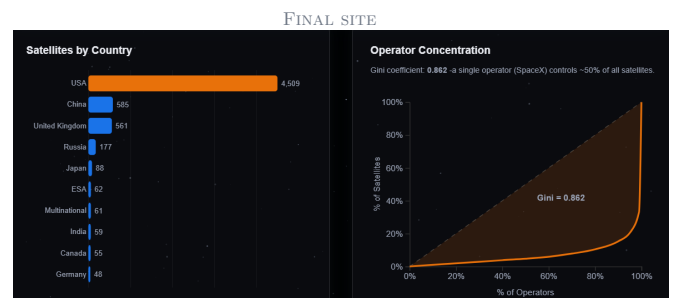
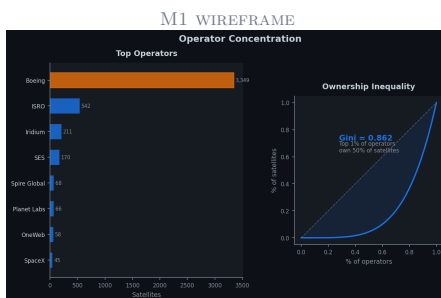
Where we started, where we landed

The M1 problematic was deliberately broad: “orbital space is becoming infrastructure, and that infrastructure is invisible to most people.” We picked the UCS Satellite Database because it combines what a satellite is, who operates it, what it is for, and where it sits in a single curated file. Three things changed between the M1 sketches and the final site: the thesis sharpened, the chapter count grew from three to four, and the 3D component was promoted from a Chapter 3 prop to the page’s hero. The three M1 wireframes below are shown next to their final-site counterparts so the evolution is visible at a glance; a fourth element (the 3D hero) had no M1 sketch and is captured separately on page 1.

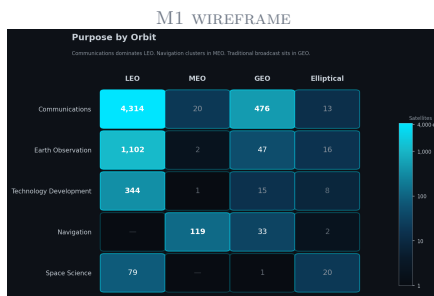
Wireframe 1 → Chapter 1. Kept almost unchanged: five scroll-locked steps still anchor the timeline at 1974, 1998, 2018, 2021, 2023. The single addition is a stacked-area panel underneath so the reader sees that the post-2019 surge is overwhelmingly Communications (Starlink and OneWeb).



Wireframe 2 → Chapter 2. The Lorenz + Gini 0.862 callout survived as drawn (the wireframe used a placeholder ranking of operators; the final swaps in the real top-10 with SpaceX dominating). Around it we added a world choropleth, a country bar, an operator deep-dive drawer, and a country×purpose specialization heatmap.



Wireframe 3 → Chapter 3. The heatmap stayed exactly as designed because it was already doing what a heatmap should: making a 2D conditional distribution legible. The static globe sketch was replaced by a Globe.gl viewport with three composed filters (orbit, year, mega-constellation) and a satellite-name search with a pulsing-ring spotlight; that interactive surface is documented on page 6.



What wasn't in M1. A 3D hero (page 1) and a fourth chapter. M1's narrative ended on present-day crowding, but the 2030 projections are what give the story stakes, so Chapter 4 (“The Next Decade”) was added late. The hero scene was driven by the screencast: a cinematic establishing shot in the first ten seconds lets us say more, faster, than any chart could.

6,713 satellites, cleaned and aggregated

The dataset is the [UCS Satellite Database](#) (Union of Concerned Scientists), snapshot 1 January 2023, with 27 attributes per satellite. Of the 6,718 raw rows, 5 are dropped during cleaning, leaving 6,713 satellites that the site ships with. The full data path lives in `scripts/build_data.py` (about 700 lines, 23 unit + integration tests in `test_build_data.py`, all passing). Every transformation it applies is logged to [data/data_quality_flags.md](#) so any reader can audit the decisions.

What the script does

Parses numerics. Apogee, perigee, eccentricity, inclination, period. UCS ships these as strings with embedded commas; we coerce, drop rows where coercion fails on parameters we need.

Reclassifies orbit class by physics, conservatively. The UCS “Class of Orbit” column is the source of truth except where it is unambiguously contradicted by altitude or eccentricity: LEO/MEO satellites whose mean altitude is above 30,000 km override to GEO (catches Angosat-2 and WFOV Testbed), “Elliptical” satellites with near-zero eccentricity override to the right shell by altitude (catches Falconsat-7, Gaofen 11, Hisaki), and GEO satellites far from geostationary altitude with near-zero eccentricity drop back to LEO or MEO. We do not override satellites with genuinely moderate eccentricity, even when a naive threshold would flip them.

Repairs eccentricity sign-error typos. A handful of UCS rows record eccentricity outside $[0, 1)$ (for example $5.11\text{E}+02$ instead of $5.11\text{E}-02$). We discard the bad value and recompute from perigee/apogee for classification, but log the original so it’s traceable.

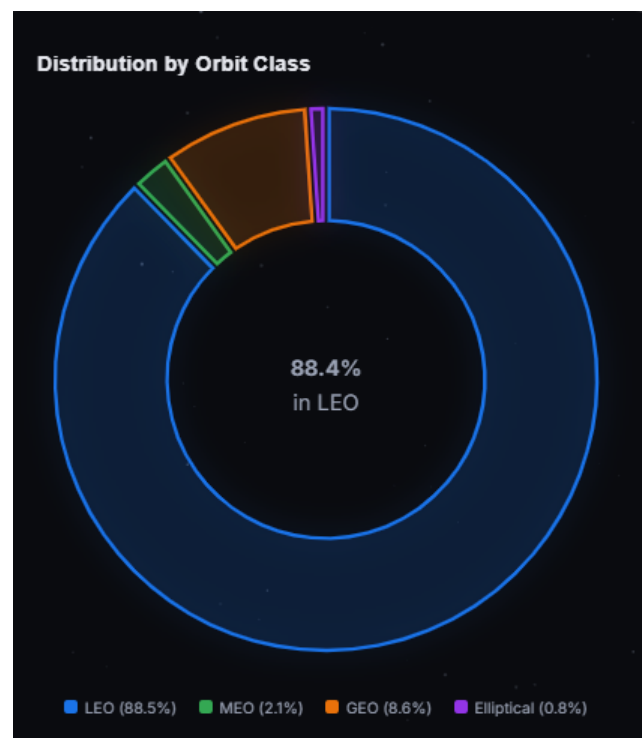
Drops irrecoverable rows. Five rows have either no inclination, no perigee *and* no apogee, or perigee > apogee (a physical impossibility). These cannot be placed on the 3D globe and are excluded from both the aggregate counts and the per-satellite list so the two outputs always agree on the total.

Normalizes operator names. “SpaceX” and “SpaceX,” “Iridium Communications, Inc.” and “Iridium,” a dozen others. We collapse them in a small lookup so the Chapter 2 operator chart isn’t full of duplicates.

Derives constellation. Starlink, OneWeb, Iridium, Galileo are detected by case-insensitive name pattern (e.g. names starting with “Starlink”). This drives the Chapter 3 mega-constellation isolator. Final distribution: Starlink 3,395, OneWeb 502, Iridium 73, Galileo 28, Other 2,715.

The picture that justified the entire project

The orbit donut from Chapter 3 (right) shows why the four-bucket classification is meaningful, not arbitrary: the population concentrates sharply. LEO holds **5,940 satellites (88.4%)**, almost an order of magnitude more than every other orbit combined. GEO accounts for **580 (8.6%)**, MEO for **142 (2.1%)**, and the highly-elliptical category for **51 (0.8%)**. The buckets correspond to physically distinct altitude bands (LEO below 2,000 km, MEO between 2,000 and $\approx 36,000$ km, GEO at $\approx 36,000$ km, Elliptical with high eccentricity) where satellites cluster for reasons of orbital mechanics, launch economics, and mission requirements. This concentration is the structural foundation of every other chart on the site: the purpose $\times$ orbit heatmap, the time-slider history, the bloom-ringed shells in the hero.



Output schema

The script writes two JSON files. `satellites.json` carries pre-aggregated statistics keyed by chart: `cumulative_launches`, `cumulative_purpose_by_year`, `top_countries`, `country_counts`, `country_purpose_matrix`, `orbit_distribution`, `purpose_by_orbit`, `top_operators`, `operator_details`, `lorenz_curve`. `satellites-globe.json` is the slim per-satellite list (name, country, orbit class, purpose, altitude, inclination, launch year, constellation) consumed by the

3D scene and Globe.gl. The `--check` mode verifies the committed JSONs match what the script would currently emit, so a stale build cannot accidentally land.

A four-chapter arc, Freytag inside

The defining decision was to commit to a *linear* reading experience and let interactivity bloom from inside it, rather than build a dashboard the user has to assemble in their head. The visual reference was The Pudding’s scrollytelling, the structural reference was Freytag’s pyramid (Lecture 12). The reader does not need a hypothesis or a query to engage. They scroll, and a story unfolds. They can pause anywhere to interact.

Freytag’s pyramid mapped to chapters

Phase	Chapter	What it does
Exposition	Hero	A black space with three sequential lines of poster text (<i>A frontier. Then, an avalanche. 6,713 and counting.</i>), then a radial-iris wipe reveals the 3D Earth surrounded by the satellite cloud. Sets the scene with a single image.
Rising action	Chapter 1, The Explosion	Cumulative-launches timeline paced by five scroll steps (1974, 1998, 2018, 2021, 2023), then a stacked-area showing how Communications absorbed the entire post-2019 growth.
Climax	Chapter 2, Who Owns Space?	The inequality reveal: USA owns two thirds, one company owns half, Gini 0.862. World choropleth, country bars, Lorenz curve, top-10 operators with deep-dive, country×purpose specialization heatmap.
Falling action	Chapter 3, The Congestion	The reader is given the keys: 3D globe with composed filters (orbit, year, constellation), year-history playback, satellite name search with spotlight. The same 6,713 dots from the hero are now theirs to inspect.
Resolution	Chapter 4, The Next Decade	2023 vs. $\approx$ 2030 filing totals. A 12× growth bar. Closes the arc without preaching.

Why these chart types, specifically

Area chart over a bar chart for the timeline (L6, L11). Cumulative counts year-by-year are a continuous quantity; bars would imply discrete events. Area fills the space below the line, which gives the chapter its second visual finding: the area under the curve is mostly post-2019.

Stacked area for purpose-over-time (L4, L6). Showing six absolute counts on a shared X axis would clutter; a single stacked area reads as “what the satellites were for, year by year” at a glance. The order (Other at the bottom, Communications on top) is chosen so the blue Communications band sits where its post-2019 explosion is most visible.

World choropleth for country distribution (L8). The country bar chart ranks (which is what we want for an “inequality” chapter), but a map gives the geographic intuition that a list cannot. Equal Earth projection because Mercator would give Greenland and Antarctica visual weight they don’t earn.

Lorenz curve over a Pareto-style chart (L6, L11). The Lorenz curve is the canonical inequality visualization, and the Gini coefficient is its single number. Pairing the curve with the 0.862 annotation makes the math legible without a stats degree.

Country×purpose heatmap over a stacked bar (L6, L11). A heatmap shows *specialization*, not just volume: the USA-Communications cell glows bright not because the USA has the most satellites, but because Communications is what it focuses on. A stacked bar would hide that pattern in the proportions.

3D globe over a 2D world map (L8). Real altitude matters for the orbit story. A 2D map projection erases altitude. Globe.gl gives us altitude, drag-rotate, and per-point hover with the right framing for the screencast.

A dark space, four hues, two typefaces

The site is a single visual system. The same palette, type, and motion grammar runs from the hero through Chapter 4 and the stats bar. The goal was that any screenshot of the site should be recognizable as the same project, regardless of which chapter it comes from.

Color

Four orbit-class hues form the spine: **blue** `#1A73E8` for LEO, **orange** `#E8710A` for MEO, **green** `#34A853` for GEO, **purple** `#9334E6` for Elliptical. The same hex codes are used identically in the D3 charts, the Three.js hero scene's GLSL shader, and the Chapter 3 globe.gl points layer, defined once in `prototype/js/charts/shared.js` and imported wherever a chart needs the palette. The dual role (orbit class *and* accent / highlight) is the one place where the palette had to flex: in the country bar chart, the USA bar uses the orange accent because it is the dominant entry, which conflicts with MEO's orange. We accepted the conflict because the rule "the dominant entry is orange" generalizes cleanly across charts (it is also the highlight for active constellation isolator buttons), whereas a sixth color would have diluted the orbit-class palette.

Typography

Two families. **Space Grotesk** (700, 800) for headings, big numbers, and any monospace-feeling moment (year readouts, KPI tiles, axis labels for the operator drawer). **Inter** (300, 400, 600) for everything else: body copy, captions, button labels, tooltips. Sizes step in a 1.25 modular scale. `tabular-nums` is on for any column of numbers that the reader will visually align.

Dark theme

A black-to-near-black background (`#000` body, `#050508` for sections) is doing three things at once: it lets the 3D scene feel like real space without a black-rectangle frame, it gives the satellite-cloud bloom and the gradient charts maximum contrast, and it lowers the energy cost of a long-running animation on the user's screen. The risk is text legibility; we mitigate with the secondary text color (`#9BA3B5`), generous line height (1.6 body, 1.7 chart steps), and an explicit `prefers-color-scheme` that doesn't fight the OS.

Motion

Two principles, applied consistently. **(1) Motion is informative or it is gone.** Every transition either communicates state change (a constellation isolator button activating, a slider thumb moving, a country bar growing in on mount) or it is removed in `prefers-reduced-motion`. The hero intro acts, the cloud orbital motion, the chapter-entrance fades, and the Globe.gl autorotation are all gated on the same media query. **(2) Easing matches the cadence of reading.** The repeated curve `cubic-bezier(0.22, 1, 0.36, 1)` (a soft ease-out) is used for chart entrances, slider thumb scale, drawer slide, fade-in opacities, and Lenis smooth-scroll. The interval (350–850 ms) sits in the range where the eye registers the motion without waiting for it.

Library choices, with rationale

D3 v7 for every 2D chart. Familiar from the lectures (L4, L5), gives us full layout control, hooks into the existing palette tokens directly. We did not reach for higher-level chart libraries (Vega-Lite, Plotly) because the project's interaction model is non-standard (composed filters, linked views, a slide-in drawer) and we wanted no friction at the boundary.

Globe.gl for the Chapter 3 globe. It is a thin wrapper around three-globe / Three.js but with a clean declarative API (`pointsData`, `ringsData`, `pointOfView`), built-in atmosphere and texture loaders, and a working camera controller. Writing this from scratch in raw Three.js would have cost two days and shipped strictly worse.

Three.js (r170) directly for the hero scene, because we needed full control: per-vertex GLSL shader on the satellite cloud, real Kepler positions updated 60 times a second, a custom `UnrealBloomPass` composer, sRGB-correct output via `OutputPass`. We ship Three.js as an ES module from `esm.sh` so the post-processing example modules resolve their bare `"three"` imports to the same Three.js instance, avoiding the "two Three.js" class-identity bug that bit us during integration.

TopoJSON + world-atlas for the choropleth. 100 KB of vector geometry, decoded with `topojson-client`. We considered raster but the map needs hover hit-testing on every country.

Lenis for smooth scroll, **D3.js** for every other transition. We deliberately did not bring in GSAP. Lenis owns the scroll, `IntersectionObserver` drives the fade-ins, D3 transitions drive every chart entrance. The dependency surface stays small.

Pre-aggregated JSON over in-browser CSV parsing. `build.data.py` ships two tight JSONs (about 200 KB combined, gzipped to about 40 KB). Parsing the 6,700-row UCS CSV on every page load would add hundreds of milliseconds to first paint and dump a megabyte over the wire. The build script is the right place for the aggregation, the per-chart shape, and the data-quality logging.

Six interactions, three problems that nearly bit us

The interactions, in the order the reader meets them

- 1. Cinematic hero intro.** Three poster lines (“A frontier.” → “Then, an avalanche.” → “6,713 and counting.”) crossfade over a black vignette; a radial-iris wipe reveals the 3D scene; the CROWDED ORBIT title slides in. One-shot: latched after `final`, so scrolling back up does not replay it. The phase controller (`cinematic-hero.js`) is a tiny state machine driven by wheel/touch input that re-emits to the rest of the page only after the latch.
- 2. Scrollytelling timeline (Chapter 1).** Native `IntersectionObserver`-style scroll handler dispatches a `scrolly-step` event when the active step changes; the area chart listens and updates a clip-path width plus an annotated marker. Five steps, five known years, deterministic.
- 3. Operator deep-dive drawer (Chapter 2).** Click any of the top 10 operator bars and a 480 px panel slides in from the right with the operator’s launch sparkline, orbit-class horizontal bars, purpose horizontal bars, and KPI tiles. Closeable via the `×` button, the dim scrim, or Escape. Body scroll is locked while open (`lenis.stop()`).
- 4. Composed filters on the Chapter 3 globe.** Three independent filters apply at once: orbit-class buttons (LEO / MEO / GEO / Elliptical / All), mega-constellation isolator (Starlink / OneWeb / Iridium / Galileo / Other / All), and a year slider with a play-history sweep (≈ 3 years/sec). A single `applyFilters()` composes all three. Selecting a constellation also dispatches a `constellation-changed` event that the world map and the country bar chart subscribe to: the home country lights up on both, linked-view style (L5).
- 5. Satellite name search with pulsing-ring spotlight.** Live filter over all 6,713 names. Ranking is prefix-match first, then shorter name, then lexicographic; so “hubble” surfaces the Hubble Space Telescope as the top hit, “iss” surfaces the ISS, “star” opens onto the Starlink fleet. Picking a result flies the Globe.gl camera to the satellite’s position and `ringsData` pulses a white ring at that point’s altitude.
- 6. Future projection (Chapter 4).** A two-bar comparison on the same X scale (so the lengths are visually honest) with five colored segments for the projected 2030 fleet. A collapsed Sources block under the chart cites every filing source (FCC, ITU, UK Space Agency).

Challenges, with what we did about each

The two-Three.js bug. Adding the hero’s Three.js scene alongside Globe.gl initially crashed Globe.gl with `z0.Timer is not a constructor`. Globe.gl bundles its own Three.js internally, and our external Three.js (loaded as a UMD script tag) overrode `window.THREE` with an older version that lacked `THREE.Timer`. Fix: load the hero’s Three.js as an ES module from `esm.sh` so it has its own isolated instance, never touches `window.THREE`, and Globe.gl keeps its own bundled copy. The two instances coexist without conflict.

Rendering 6,713 animated satellites at 60 fps. Naive Three.js (one `Mesh` per satellite) would have been a 6,700-draw-call disaster. We use a single `BufferGeometry` with one vertex per satellite, custom `ShaderMaterial` (vertex shader handles per-orbit point size and depth attenuation, fragment shader draws each as a soft glowing disk with additive blending), and a tight per-frame loop that updates only the `position` attribute. CPU cost is roughly 30 floating-point ops per satellite per frame (≈ 200 K total) and the GPU upload is ≈ 80 KB per frame. Comfortable margin on a 2019 laptop.

Operator-name normalization. The UCS CSV writes “Spacex,” “SpaceX,” “Space Exploration Technologies,” and “Space Exploration Technologies Corp. (SpaceX)” for the same operator. We collapse those (and 17 others) in a static lookup table in `build.data.py`. Without this, the top-10 operator chart would show three SpaceX entries.

The Chapter 1 sticky-chart on tall content. The scrollytelling chart uses `position: sticky` and `min-height: 300vh` on its container so the chart sticks for the full scroll height. When we added the second panel (purpose-over-time) inside the same chapter, the sticky rule applied to it too and created a giant dark gap. Tightening the selector to `:has(.scrolly)` scoped the rule to the right container only.

Build, ship, test

Architecture

The site is fully static. A Python build script transforms one CSV into two JSONs, the browser fetches them, every chart renders client-side. No backend, no build step on the client. The data pipeline is the only non-trivial moving part:

```
UCS CSV → build_data.py → satellites.json + satellites-globe.json → fetch() → D3 / Three.js /
                                           Globe.gl
```

The Python side enforces correctness (23 unit + integration tests, all green); the JS side stays dumb because the data shape is already right.

Module layout

`prototype/js/` is organized into four buckets. **Charts** (`charts/*.js`, one file per chart) each export a top-level draw function and share constants from `shared.js` (palette, tooltip). **Scene** (`scene/journey.js`) is the hero's Three.js scene, loaded as an ES module with its own Three.js instance. **Effects** (`effects/stars.js`) are the page-wide canvas decorations. **Controllers**: `main.js` (boot), `globe.js` (Chapter 3 Globe.gl + filters + search), `scrollytelling.js` (Chapter 1 step observer), `scroll.js` (Lenis + chapter fade-ins), `nav.js` (chapter navigator), `cinematic-hero.js` (intro state machine).

Imports stay flat (no bundler, no transpiler). Two files are ES modules because they need ES imports (`scene/journey.js` for Three.js, `scroll.js` for Lenis); everything else is a plain global `<script>`, in deterministic load order, with `shared.js` first. This is intentional: the team can clone the repo, run a Python HTTP server, and edit any chart with no toolchain.

Build pipeline and testing

`scripts/build_data.py` owns the full transformation. `--check` mode recomputes both JSONs in memory and compares against the committed files, exiting non-zero on drift. `scripts/test_build_data.py` runs 23 tests: orbit-classification rules, eccentricity-sign-error recovery, purpose normalization, operator-name normalization, sanity-check flagging, end-to-end build against the real CSV, JSON validity. The whole pipeline runs in about 1.5 seconds; the tests in < 1 second.

Performance

The site loads in <1.5 s first paint on a 4G connection. The two JSONs gzip to ≈ 40 KB, Three.js is the largest dependency (≈ 150 KB gzipped, loaded as an ES module). The 6,713-satellite scene renders consistently at 60 fps on integrated-graphics laptops; we verified on a 2019 MacBook Air and a Windows-on-ARM Surface. The Chapter 3 Globe.gl viewport hovers around 55–60 fps depending on autorotate. `pointsMerge(true)` keeps the 6,713-point cloud as a single GPU mesh.

Accessibility

Every interactive control (filter buttons, year slider, play button, search input, drawer close, constellation buttons) has an `aria-label` and is keyboard-reachable. The operator drawer is a real `role="dialog"` with `aria-modal` and focus management on open. Globe.gl content has an `aria-label` description of the visual so screen readers do not just see “canvas.” A `<noscript>` block tells JS-disabled visitors what they are missing and how to fix it. The whole site degrades under `prefers-reduced-motion`: Lenis is not constructed, fades land immediately, the hero's orbital motion and the globe's autorotation freeze. Color contrast on body text is > 7:1, on secondary text > 4.5:1.

Responsive

Three breakpoints (700 px, 900 px, viewport-bound). The country bar / Lorenz / operator / heatmap row collapses to single-column under 900 px. The time-slider card stacks the play button above the track under 700 px. The operator drawer takes 100 vw on phones. The 3D globe and the hero scene scale via `ResizeObserver` on their containers, not on window resize, so late-arriving layout (fonts loading, scrollbar appearing) does not leave them at 0 px wide.

Browser support

Tested on Chrome, Firefox, Safari (desktop and iOS), Edge. The CSS uses `:has()` for one selector (Chapter 1 sticky scoping), which lands in every major browser shipped since late 2023. Earlier browsers see a static, non-immersive version of the site that still works because every chart degrades gracefully without the JS post-processing.

Who did what, and what we would do differently

The contributions below reflect the actual commit history and pairing during the working sessions of milestone 3. Every member contributed across the stack; the breakdown captures where each took primary ownership.

Member	Primary contributions	Share
Kevin Abou Jaoude(358300)	Data pipeline expansion in <code>build.data.py</code> : new aggregations for cumulative-purpose-by-year, country counts, country×purpose matrix, operator deep-dive details, plus the constellation and launch-year fields on the per-satellite list (23 tests passing throughout). Chapter 4 (“The Next Decade”) projection chart with the citation block. Lenis smooth-scroll integration and the chapter-entrance fade-in observer. Site-wide accessibility pass (<code>aria-labels</code> , <code>noscript</code> fallback, <code>prefers-reduced-motion</code> guards, Open Graph meta). Repo hygiene: README v2, MIT LICENSE, expanded <code>.gitignore</code> . Process-book authoring and structure.	1/3
Youssef Dib(339964)	Cinematic hero intro: three-act poster sequence, radial-iris vignette wipe, latched phase state machine that consumes wheel/touch input without leaking page scroll. Fixed top-right chapter navigator (open / close / active-chapter tracking via <code>IntersectionObserver</code>). Hero 3D scene polish: <code>UnrealBloomPass</code> post-processing, custom GLSL shader for soft-glow satellite sprites, orbit-class color tuning so each colored shell reads against the LEO mass. Globe.gl robustness: Ctrl-to-zoom gating with on-screen hint, <code>ResizeObserver</code> pattern, atmosphere alpha fix, scrollbar hiding so the layout never shifts.	1/3
Mark Nabbout(358312)	Chapter 1 stacked area chart (purpose over time) with the post-2019 Starlink-era annotation. Chapter 2 world choropleth (D3 + TopoJSON, Equal Earth, log color scale, country-name mapping). Chapter 2 country×purpose specialization heatmap with floating info card. Chapter 2 top-operators bar chart and full deep-dive drawer (launch-timeline sparkline, orbit and purpose mini-bars, KPI tiles, focus-trapped <code>role="dialog"</code>). Chapter 3 year-filter slider with play-history sweep, mega-constellation isolator with linked-view dispatch, satellite name search with pulsing-ring spotlight. Linked-country highlighting wiring across the world map and the country bar chart.	1/3

What surprised us

The data was less linear than expected. M1 framed the story as “orbit is filling up.” That’s true, but the Chapter 1 stacked area showed something sharper that we did not anticipate: it is filling up *with one purpose*. Communications was already 40% of the fleet in 2018; by 2022 it was 72%. The chapter changed shape because we found that finding.

Performance was not the bottleneck. We expected 6,713 animated satellites at 60 fps to be a fight. The custom shader handled it on the first try because the GPU is doing exactly the thing it is good at (one vertex per satellite, one position update per frame, additive-blended quad sprites). The bottlenecks we actually hit were CDN-related: a Lenis URL that 404’d silently, a Three.js post-processing module trying to bare-import “`three`” without an import map.

Honest data caveats land harder than we feared. We worried that the Chapter 4 “filings are not deployments” caveat would deflate the punch of the 12× headline. It didn’t. Putting the citations in a collapsed expandable block, with a single warning line about deployment slippage, came off as confident rather than hedging.

What we would do differently with more time

Real ephemeris. Our 3D scene places satellites on inclination-bounded *visual* positions, not their real lat/lng at a given instant. With real two-line-element data from CelesTrak, the hero scene could show satellites where they actually are right now. The cost is a recurring data fetch and a `satellite.js` dependency. Worth doing as a v2.

A debris and conjunction layer. The story stops at active satellites. The full story includes the $\approx 30,000$ tracked debris objects and a Kessler-syndrome explainer. That is its own chapter and its own dataset (NORAD), but the same machinery would carry it.

A guided auto-tour mode. A play button at the top of the page that scripts a camera flythrough across all four chapters, so a first-time visitor with no idea what scrollytelling is gets the entire story in 90 seconds without scrolling. Effectively the screencast, but built into the page.

Localization. The narrative is in English. Translations would not change the data; they would change the audience. Most of the strings live in one HTML file, so this is engineering, not research.

Sound. A subtle ambient layer with a mute toggle, off by default. Risky to do well, immediate to do badly. We chose to do nothing rather than do it half-way; with more time we’d try it.